



西安交通大学
XI'AN JIAOTONG UNIVERSITY

机器学习

实验报告

基于 EfficientNet-B0 的海贼王主要 角色识别模型

2024-2025 学年 第二学期

学 院： 数学与统计学院
班 级： 数学强基 2202
学 号： 2224110096
姓 名： 陈宇昂
指导教师： 孟德宇老师
实验地点： 数学实验中心
完成时间： 2025-03-14

目录

1	实验内容	1
1.1	问题描述	1
1.2	实验步骤	1
1.3	实验要求	1
2	实验描述	1
2.1	数据集与实验环境参数	1
2.2	算法描述	2
2.3	理论推导	3
2.3.1	EfficientNet 复合缩放公式	3
2.3.2	EfficientNet-B0 整体架构	4
2.3.3	交叉熵损失函数	5
2.3.4	Adam 优化算法	5
2.4	模块化设计	5
2.4.1	数据集模块	5
2.4.2	模型的建立	6
2.4.3	可视化	6
2.4.4	模型的训练和评估	6
2.5	正则化	7
2.5.1	数据增强	7
2.5.2	权重衰减	7
2.5.3	早停机制	7
2.6	程序框图	8
3	实验结果与所遇问题	8
3.1	实验结果分析	8
3.2	问题与解决措施	10
A	附录：实验代码	12
A.1	Project 的目录结构	12
A.2	EfficientNet-B0.py	12

1 实验内容

1.1 问题描述

本实验旨在构建一个海贼王主要角色识别模型，在小样本数据集（各类别均约 650 条数据）与低算力设备（NVIDIA GeForce RTX 3050 Ti）的限制下，于合理时间内能实现对模型的训练，并能够对测试集中路飞、索隆等 5 个核心角色尺寸不一的图片进行高精度分类。

1.2 实验步骤

1. 数据准备：收集数据集，统一图片格式；
2. 数据预处理：统一训练集验证集的图像尺寸，划分训练集/验证集；
3. 模型构建：调用 EfficientNet-B0 模型进行迁移学习；
4. 训练优化：采用 Adam 优化器对模型进行优化；
5. 模型评估：对测试集中的分类结果进行人工验证，计算出其准确率；
6. 结果分析：对比不同超参数组合的结果，找出最适宜的参数组合。

1.3 实验要求

1. 模型在测试集上的分类准确率 $\geq 90\%$ ；
2. 模型能够在合理时间范围内 ($< 10\text{min}$) 完成训练并输出结果；
3. 选取合适的超参数以平衡模型训练时间和分类准确率之间的关系。

2 实验描述

2.1 数据集与实验环境参数

训练集与验证集 本次实验的训练集与验证集来自于Kaggle 用户 Ibrahim Serouis 99。原数据集中包含艾斯 (Ace)、赤犬 (Akainu) 等 18 个海贼王中的主要角色，由于算力有限，本人按照草帽海贼团的上船顺序选择路飞 (Luffy,654 张)、索隆 (Zoro,659 张)、娜美 (Nami,665 张)、乌索普 (Usopp,658 张)、山治 (Sanji,651 张) 这五位角色的图片作为本次实验的训练集与验证集。

测试集 使用 Edge 浏览器搜集 5 个角色各 8 张图，将图片格式统一转换为 jpg。

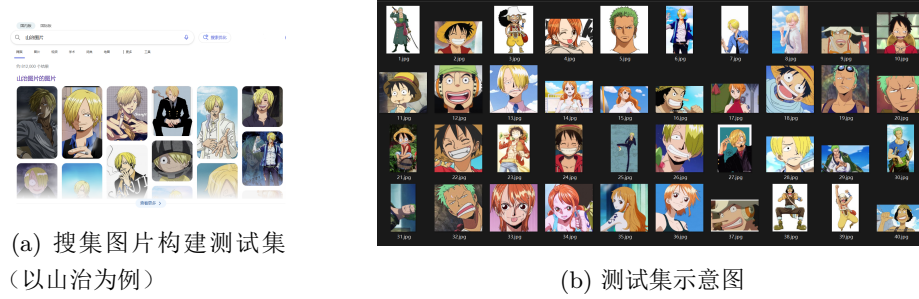


图 1: 构建测试集

实验环境参数 本次实验的相关参数如表1所示。

表 1: 实验运行相关参数

项目	参数
执行平台	PyCharm
主要依赖库	Pytorch, Sklearn, Numpy
Python 版本	Python 3.11.8
GPU 型号	NVIDIA GeForce RTX 3050 Ti
CUDA 版本	CUDA 12.7

2.2 算法描述

传统 CNN 模型缩放方法（如增加深度或宽度）存在两个明显的效率瓶颈：第一，单一维度的缩放会导致准确率增长饱和；第二，手动调整两或三个维度需大量试错，且易陷入次优解，无法充分利用计算资源。

	传统设计	放缩方法
层结构 $\mathcal{F}_i$	搜索最优操作（如卷积类型、核大小）	固定 $\mathcal{F}_i$ （如 MBConv）
深度 L_i	设计阶段重复次数	$L_i = \alpha^\phi \cdot \hat{L}_i$
宽度 C_i	手动调整通道数	$C_i = \beta^\phi \cdot \hat{C}_i$
分辨率 H_i, W_i	固定输入尺寸	$H_i, W_i = \gamma^\phi \cdot \hat{H}_i, \hat{W}_i$

在计算资源吃紧的情况下，我选择使用 Google 科学家 Mingxing Tan 和 Quoc V. Le 于 2019 年提出的 EfficientNet-B0 模型 [1]。该模型使用了一种平衡缩放网络深度、宽度和输入分辨率的复合放缩策略以充分利用计算资源。

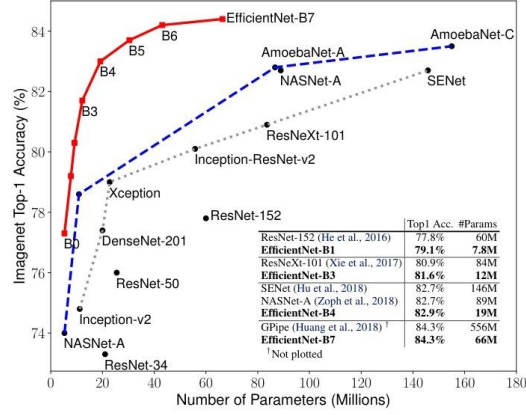


图 2: 模型参数大小与 ImageNet 上准确率散点图

2.3 理论推导

2.3.1 EfficientNet 复合缩放公式

目标函数

$$\max_{d,w,r} \text{Accuracy}(\mathcal{N}(d, w, r))$$

其中 d, w, r 分别为深度、宽度、分辨率系数， $\mathcal{N}(d, w, r)$ 表示在此参数组合下输出的结果。

约束条件

$$\begin{cases} \text{FLOPS}(\mathcal{N}) \propto \sum_{i=1}^s L_i \cdot C_i^2 \cdot H_i \cdot W_i \propto d \cdot w^2 \cdot r^2 \leq \text{target_flops} \\ \text{Memory}(\mathcal{N}) \propto \max_i (H_i \cdot W_i \cdot C_i) \leq \text{target_memory} \end{cases}$$

其中，Memory 表示内存，FLOPS 表示总计算量。

缩放公式

$$\begin{aligned}
\mathcal{N}(d, w, r) &= \bigodot_{i=1}^s \mathcal{F}_i^{L_i}(X_{\langle H_i, W_i, C_i \rangle}) \\
L_i &= d \cdot \hat{L}_i, \quad C_i = w \cdot \hat{C}_i, \quad H_i \times W_i = r \cdot (\hat{H}_i \times \hat{W}_i) \\
d &= \alpha^\phi, \quad w = \beta^\phi, \quad r = \gamma^\phi \\
\text{s.t. } &\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2 \\
&\alpha \geq 1, \beta \geq 1, \gamma \geq 1
\end{aligned}$$

其中， Y_i 表示第 i 层的输出张量； X_i 表示输入张量，形状为 $\langle H_i, W_i, C_i \rangle$ ； $\odot$ 表示层操作符（如卷积、激活函数、池化等）

系数确定 根据论文 [1] 中的结果，可知

$$\alpha = 1.2, \quad \beta = 1.1, \quad \gamma = 1.15 \quad \text{满足} \quad \alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$$

2.3.2 EfficientNet-B0 整体架构

1. **初始卷积层**：快速下采样（224→112），提取低级特征
2. **MBConv 模块**：深度可分离卷积减少参数量、扩展-投影结构平衡计算效率、跳跃连接提升梯度流动
3. **SE 模块**：压缩（全局平均池化）、激励（全连接层）、通道加权
4. **复合缩放**：同步调节深度宽度分辨率
5. **分类层**：全局平均池化（ $7 \times 7 \rightarrow 1 \times 1$ ），全连接层输出类别概率

表 2: EfficientNet-B0 网络结构

阶段 i	操作 $\hat{\mathcal{F}}_i$	分辨率 $\hat{H}_i \times \hat{W}_i$	通道 $\hat{C}_i$	层数 $\hat{L}_i$
1	Conv3x3	224×224	32	1
2	MBConv1, k3x3	112×112	16	1
3	MBConv6, k3x3	112×112	24	2
4	MBConv6, k5x5	56×56	40	2
5	MBConv6, k3x3	28×28	80	3
6	MBConv6, k5x5	14×14	112	3
7	MBConv6, k5x5	14×14	192	4
8	MBConv6, k3x3	7×7	320	1
9	Conv1x1 & Pooling & FC	7×7	1280	1

2.3.3 交叉熵损失函数

交叉熵损失函数用于衡量模型预测结果与真实标签之间的差异。对于多分类问题，交叉熵损失函数可以表示为：

$$L = - \sum_{i=1}^N \sum_{k=1}^K y_{i,k} \log(p_{i,k})$$

其中， N 是样本数量， K 是类别数量， $y_{i,k}$ 是第 i 个样本的第 k 个类别的真实标签（通常为 0 或 1）， $p_{i,k}$ 是模型对第 i 个样本的第 k 个类别的预测概率。

交叉熵损失函数的目标是使模型的预测概率尽可能接近真实标签，当模型的预测概率与真实标签完全一致时，交叉熵损失函数的值为 0。

2.3.4 Adam 优化算法

Adam 优化算法是一种自适应学习率的优化算法，它结合了 AdaGrad 和 RMSProp 的优点。Adam 算法通过计算梯度的一阶矩估计（均值）和二阶矩估计（方差），自适应地调整每个参数的学习率。

Adam 算法的更新公式如下：

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t} \\ \hat{v}_t &= \frac{v_t}{1 - \beta_2^t} \\ \theta_{t+1} &= \theta_t - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t \end{aligned}$$

其中， m_t 和 v_t 分别是梯度的一阶矩估计和二阶矩估计， β_1 和 β_2 是衰减率， g_t 是当前时刻的梯度， $\hat{m}_t$ 和 $\hat{v}_t$ 是修正后的一阶矩估计和二阶矩估计， α 是学习率， ϵ 是一个很小的常数，用于避免分母为 0， θ_t 是当前时刻的参数。

2.4 模块化设计

2.4.1 数据集模块

数据集模块主要负责数据的加载和处理。

1. AnimeDataset 类:
 - (a) 初始化时接收图像路径列表、标签列表和数据转换函数作为参数;
 - (b) 实现 `__len__` 方法, 返回数据集的样本数量;
 - (c) 实现 `__getitem__` 方法, 根据索引获取图像和对应的标签, 并对图像进行数据转换;
2. `prepare_dataloaders` 函数: 将数据集划分为训练集和验证集, 并创建相应的数据加载器 (DataLoader), 以实现数据的批量加载和随机打乱, 方便模型的训练和评估。

2.4.2 模型的建立

模型模块负责构建和初始化模型。在本实验中, 使用 `create_model` 函数创建 EfficientNet-B0 模型, 将模型的最后一层全连接层替换为一个新的全连接层, 其输出维度为 5, 对应于 5 个海贼王角色类别。

2.4.3 可视化

可视化模块用于对数据和模型的训练过程进行可视化展示。包括以下几个函数:

1. `plot_data_samples` 函数: 用于展示训练集中的样本图像, 包括图像的可视化和对应的标签;
2. `plot_training_curves` 函数: 用于绘制训练损失曲线和验证准确率曲线, 直观地展示模型的训练过程;
3. `plot_confusion_matrix` 函数: 用于绘制混淆矩阵, 展示模型在每个类别上的分类情况;
4. `visualize_predictions` 函数: 用于可视化模型的预测结果, 展示测试集中图像的预测标签和置信度。

2.4.4 模型的训练和评估

训练和评估模块负责模型的训练和评估。包括以下几个函数:

1. `evaluate` 函数: 用于在验证集上评估模型的准确率;
2. `train_model` 函数: 用于训练模型, 包括模型的初始化、损失函数和优化器的定义、训练循环的实现等。在训练过程中, 会调用 `'evaluate'` 函数定期评估模型的性能, 并保存最佳模型;

3. `predict` 函数：用于对单张图像进行预测，返回预测的类别；
4. `batch_rename_images` 函数：用于对测试集中的图像进行批量重命名，根据模型的预测结果修改图像的文件名。

2.5 正则化

2.5.1 数据增强

数据增强是一种常用的正则化方法，它通过对训练数据进行随机变换，增加数据的多样性，从而提高模型的泛化能力。在本实验中，使用了随机裁剪和随机水平翻转两种数据增强方法。

随机裁剪可以模拟不同的视角和尺度，使模型能够学习到不同位置和大小角色特征。随机水平翻转可以增加图像的对称性变化，使模型对图像的左右翻转具有不变性。通过数据增强，模型可以接触到更多不同的样本，减少对特定样本的过拟合。

2.5.2 权重衰减

权重衰减（L2 正则化）是一种在损失函数中添加正则化项的方法，它可以限制模型的权重大小，避免模型过拟合。在本实验中，使用 Adam 优化器时设置了 `weight_decay = 0.0001`，即在损失函数中添加了一个 L2 正则化项：

$$L_{reg} = L + \lambda \sum_i w_i^2$$

其中， L 是原始的损失函数， λ 是权重衰减系数， w_i 是模型的权重。

权重衰减的作用是在训练过程中，当模型的权重过大时，会增加损失函数的值，从而促使模型调整权重，使其变小。这样可以避免模型过于复杂，提高模型的泛化能力。

2.5.3 早停机制

早停机制是一种在训练过程中根据验证集的性能来提前停止训练的方法。在本实验中，在 `train_model` 函数中实现了早停机制。具体来说，在训练过程中，会定期评估模型在验证集上的准确率，并保存准确率最高的模型。如果在连续多个 epoch 中，验证集的准确率没有提高，则停止训练，避免模型过拟合。

早停机制的优点是可以在模型达到最佳性能时及时停止训练，节省训练时间和计算资源，同时避免模型在训练后期过拟合验证集。

2.6 程序框图

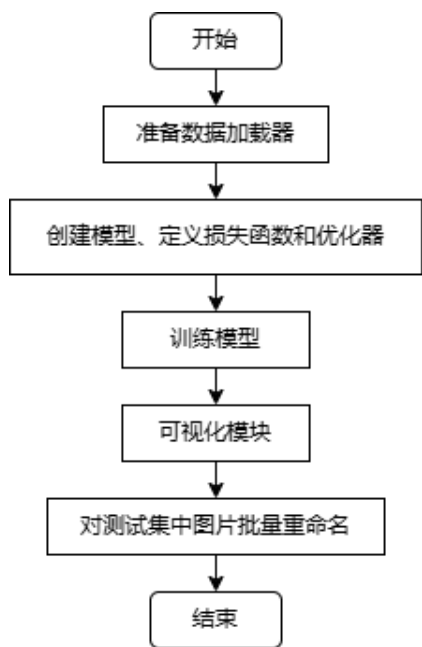


图 3: 程序框图

具体代码实现见附录。

3 实验结果与所遇问题

3.1 实验结果分析

本实验基于 EfficientNet-B0 模型构建了海贼王角色识别系统，在小样本数据集（每类约 650 张）和 NVIDIA GeForce RTX 3050 Ti 设备上完成训练。通过数据增强、迁移学习和超参数调优，最终在测试集上实现了 97.5% 的分类准确率。

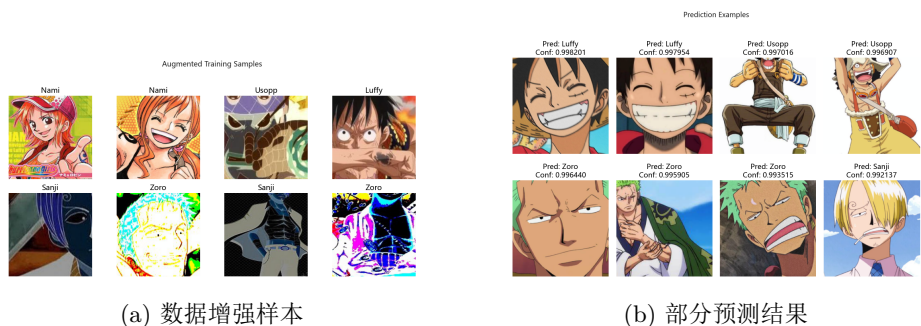


图4a展示了经过随机裁剪、水平翻转等增强后的训练样本。通过数据增强，模型对不同角度、反色情况下的角色特征具有更强的鲁棒性，有效缓解了小样本过拟合问题；图4b展示了模型的部分预测结果（图片上方的是与某类之间的相关度）。



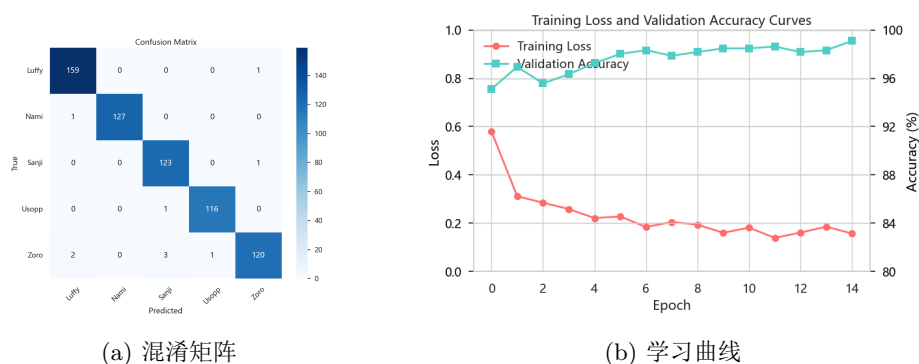
图 5: 角色识别模型在各参数下的表现

上图反映了本文的角色识别模型在各参数下的表现情况。对比图5a和图5b、图5c和图5d可以看出，batch 越大，模型最终的效果越好；对比图5b和

图5c可以看出, lr 越小, 模型的训练效果越好。

特别地, batch 越大, lr 越小, 运行时间越长, 如图5d就无法在合理时间内完成任务, 更遑论 batch=64 & lr=0.001 这一配置了。因此, 在当前情况下, 最为适宜的参数应是 batch=32 & lr=0.001 了。

图6a显示模型对路飞、索隆等 5 个核心角色在验证集上的分类效果。其中, 索隆与其它角色的混淆率最高, 说明模型对索隆 (Zoro) 的特征相对来说捕捉的较差, 这也与测试集上的结果相吻合。这可能与索隆的头发、肤色与其他男性角色的服饰、肤色撞色有关。图6b为本模型的学习曲线, 从图中可知, 模型的准确率在第 6 个 epoch 后基本趋于稳定, 在第 15 个 epoch 达到峰值 (横坐标为 14)。



3.2 问题与解决措施

事实上, 在实验初期我尝试使用 ResNet-50 作为基线模型 [2], 仅获得 82.5% 的正确率。这可能是因为 ResNet-50 模型缺乏通道注意力机制, 在小样本场景下容易出现过拟合。后来我改用 EfficientNet-B0 模型, 此模型通过引入 MBConv 模块和 SE 注意力机制, 准确率显著提升。

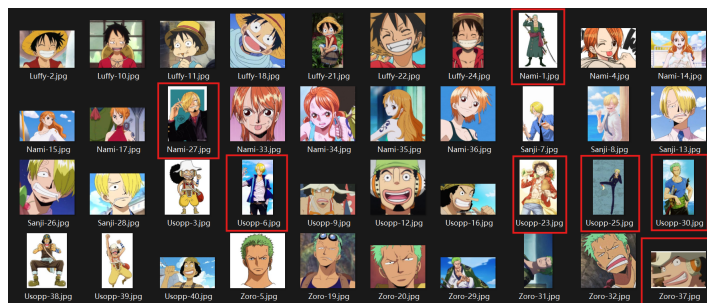


图 7: ResNet-50 在测试集上的正确率为 82.5%

参考文献

- [1] Mingxing Tan and Quoc Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In *International conference on machine learning*, pages 6105–6114. PMLR, 2019.
- [2] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

A 附录：实验代码

A.1 Project 的目录结构

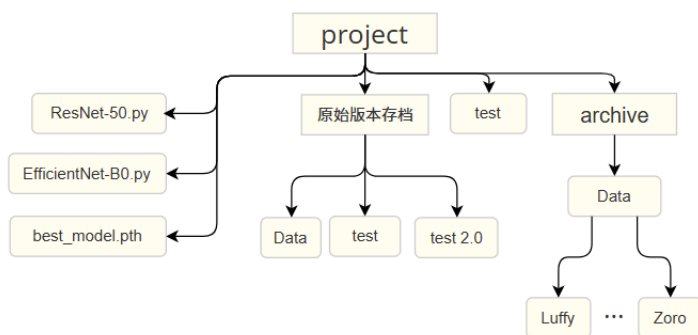


图 8: project 目录结构

A.2 EfficientNet-B0.py

参数设置（此处以 `batch_size=32`, `lc=0.001`, `num_epochs=15` 为例）

```
1 import os
2 import numpy as np
3 from PIL import Image
4 import torch
5 import torch.nn as nn
6 import torch.optim as optim
7 from torch.utils.data import Dataset, DataLoader
8 from torchvision import transforms, models, utils
9 from sklearn.model_selection import train_test_split
10 from sklearn.metrics import confusion_matrix
11 import matplotlib.pyplot as plt
12 from matplotlib.ticker import MaxNLocator
13 import seaborn as sns
14 from tqdm import tqdm
15
16 # 配置参数
17 config = {
18     "data_dir": "./archive/Data",
19     "batch_size": 32,
```

```

20     "image_size": 224,
21     "num_epochs": 15,
22     "lr": 0.001,
23     "num_classes": 5
24 }
25
26 # 可视化设置
27 plt.style.use('seaborn-v0_8')
28 sns.set_theme(style="whitegrid", font_scale=1.2)
29 try:
30     plt.rcParams['font.family'] = ['Microsoft YaHei', 'SimHei',
31                                   'sans-serif']
32 except:
33     pass
34 COLORS = ["#FF6B6B", "#4ECDC4", "#45B7D1", "#96CEB4", "#FFEEAD"]

```

数据集类:

```

1 class AnimeDataset(Dataset):
2     def __init__(self, image_paths, labels, transform=None):
3         self.image_paths = image_paths
4         self.labels = labels
5         self.transform = transform
6
7     def __len__(self):
8         return len(self.image_paths)
9
10    def __getitem__(self, idx):
11        image = Image.open(self.image_paths[idx]).convert('RGB')
12        label = self.labels[idx]
13        if self.transform:
14            image = self.transform(image)
15        return image, label
16
17
18    def prepare_dataloaders():
19        all_image_paths = []
20        labels = []
21        class_names = sorted(os.listdir(config["data_dir"]))

```

```

22
23     for label, class_name in enumerate(class_names):
24         class_dir = os.path.join(config["data_dir"], class_name)
25         for img_file in os.listdir(class_dir):
26             if img_file.endswith(('.jpg', '.png')):
27                 all_image_paths.append(os.path.join(class_dir, img_file))
28                 labels.append(label)
29
30     train_paths, val_paths, train_labels, val_labels = train_test_split(
31     all_image_paths, labels, test_size=0.2, random_state=42
32     )
33
34     train_transform = transforms.Compose([
35         transforms.RandomResizedCrop(config["image_size"]),
36         transforms.RandomHorizontalFlip(),
37         transforms.ToTensor(),
38         transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224,
39         0.225])
40     ])
41
42     val_transform = transforms.Compose([
43         transforms.Resize(256),
44         transforms.CenterCrop(config["image_size"]),
45         transforms.ToTensor(),
46         transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224,
47         0.225])
48     ])
49
50     train_dataset = AnimeDataset(train_paths, train_labels,
51     train_transform)
52     val_dataset = AnimeDataset(val_paths, val_labels, val_transform)
53
54     train_loader = DataLoader(train_dataset,
55     batch_size=config["batch_size"], shuffle=True)
56     val_loader = DataLoader(val_dataset,
57     batch_size=config["batch_size"], shuffle=False)
58
59     return train_loader, val_loader, class_names

```

模型定义:

```
1 def create_model(use_pretrained=True):
2     model = models.efficientnet_b0(pretrained=use_pretrained)
3     num_fts = model.classifier[1].in_features
4     model.classifier[1] = nn.Linear(num_fts, config["num_classes"])
5     return model
```

可视化模块:

```
1 def plot_data_samples(dataloader, class_names, n=8):
2     images, labels = next(iter(dataloader))
3     images = images[:n]
4     labels = labels[:n]
5
6     mean = np.array([0.485, 0.456, 0.406])
7     std = np.array([0.229, 0.224, 0.225])
8     images = images.numpy().transpose((0, 2, 3, 1))
9     images = std * images + mean
10    images = np.clip(images, 0, 1)
11
12    plt.figure(figsize=(12, 6))
13    for i in range(n):
14        ax = plt.subplot(2, 4, i + 1)
15        ax.imshow(images[i])
16        ax.set_title(class_names[labels[i]], color='black')
17        ax.axis('off')
18    plt.suptitle("Augmented Training Samples", fontsize=14, y=1.02)
19    plt.tight_layout()
20    plt.savefig("data_samples.png", bbox_inches='tight')
21    plt.show()
22
23 # 绘制训练损失曲线
24 def plot_training_curves(train_losses, val_accuracies):
25     plt.figure(figsize=(8, 5))
26
27     # loss
28     ax1 = plt.gca()
29     line1 = ax1.plot(train_losses, color=COLORS[0], lw=2, marker='o',
30                     markersize=8, label='Training Loss')
```

```

30 ax1.set_xlabel('Epoch')
31 ax1.set_ylabel('Loss', color='black')
32 ax1.tick_params(axis='y', labelcolor='black')
33 ax1.yaxis.set_major_locator(MultipleLocator(0.2))
34 ax1.set_ylim(0, 1)
35
36 # Acc
37 ax2 = ax1.twinx()
38 line2 = ax2.plot(val accuracies, color=COLORS[1], lw=2, marker='s',
39                  markersize=8, label='Validation Accuracy')
39 ax2.set_ylabel('Accuracy (%)', color='black')
40 ax2.tick_params(axis='y', labelcolor='black')
41 ax2.yaxis.set_major_locator(MultipleLocator(4))
42 ax2.set_ylim(80, 100)
43
44 # 合并图例
45 lines = line1 + line2
46 labels = [l.get_label() for l in lines]
47 ax1.legend(lines, labels, loc='upper left')
48
49 plt.title("Training Loss and Validation Accuracy Curves")
50 plt.tight_layout()
51 plt.savefig("training_curves.png", bbox_inches='tight')
52 plt.show()
53
54
55 def plot_confusion_matrix(model, dataloader, class_names, device):
56     model.eval()
57     all_preds = []
58     all_labels = []
59
60     with torch.no_grad():
61         for inputs, labels in tqdm(dataloader, desc="Generating
62                                     Confusion Matrix"):
63             inputs = inputs.to(device)
64             labels = labels.to(device)
65             outputs = model(inputs)
66             _, preds = torch.max(outputs, 1)
67             all_preds.extend(preds.cpu().numpy())

```

```

67         all_labels.extend(labels.cpu().numpy())
68
69     cm = confusion_matrix(all_labels, all_preds)
70
71     plt.figure(figsize=(10, 8))
72     sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
73                 xticklabels=class_names, yticklabels=class_names)
74     plt.title("Confusion Matrix")
75     plt.xlabel("Predicted")
76     plt.ylabel("True")
77     plt.xticks(rotation=45)
78     plt.yticks(rotation=0)
79     plt.savefig("confusion_matrix.png", bbox_inches='tight')
80     plt.show()
81
82
83 def visualize_predictions(test_folder, model, class_names, device, n=8):
84     model.load_state_dict(torch.load("best_model.pth"))
85     model.eval()
86
87     transform = transforms.Compose([
88         transforms.Resize(256),
89         transforms.CenterCrop(224),
90         transforms.ToTensor(),
91         transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224,
92             0.225])
93     ])
94
95     plt.figure(figsize=(12, 8))
96     count = 0
97     for filename in os.listdir(test_folder):
98         if count >= n:
99             break
100         if filename.endswith('.jpg'):
101             image_path = os.path.join(test_folder, filename)
102             try:
103                 image = Image.open(image_path).convert('RGB')
104                 input_tensor = transform(image).unsqueeze(0).to(device)

```

```

105         with torch.no_grad():
106             output = model(input_tensor)
107             probs = torch.nn.functional.softmax(output, dim=1)
108             conf, pred = torch.max(probs, 1) # 输出最大相似度的
109
110         image_np =
111             input_tensor.cpu().squeeze().numpy().transpose((1, 2,
112             0))
113
114         mean = np.array([0.485, 0.456, 0.406])
115         std = np.array([0.229, 0.224, 0.225])
116         image_np = std * image_np + mean
117         image_np = np.clip(image_np, 0, 1)
118
119         ax = plt.subplot(2, 4, count + 1)
120         ax.imshow(image_np)
121         title = f"Pred: {class_names[pred.item()]} \n Conf:
122             {conf.item():.8f}" # 相似度
123         ax.set_title(title, color='black')
124         ax.axis('off')
125         count += 1
126     except Exception as e:
127         print(f"Error processing {filename}: {e}")
128
129     plt.suptitle("Prediction Examples", fontsize=14, y=1.02)
130     plt.tight_layout()
131     plt.savefig("prediction_examples.png", bbox_inches='tight')
132     plt.show()

```

模型的训练模块:

```

1 def evaluate(model, val_loader, device):
2     model.eval()
3     correct = 0
4     total = 0
5     with torch.no_grad():
6         for inputs, labels in val_loader:
7             inputs = inputs.to(device)
8             labels = labels.to(device)
9             outputs = model(inputs)

```

```

10         _, preds = torch.max(outputs, 1)
11         correct += (preds == labels).sum().item()
12         total += labels.size(0)
13     return correct / total
14
15
16 def train_model(model, criterion, optimizer, train_loader, val_loader):
17     device = torch.device("cuda:0" if torch.cuda.is_available() else
18                           "cpu")
19     model = model.to(device)
20
21     train_losses = []
22     val_accuracies = []
23     best_acc = 0.0
24
25     plot_data_samples(train_loader, class_names)
26
27     for epoch in range(config["num_epochs"]):
28         model.train()
29         running_loss = 0.0
30         progress_bar = tqdm(train_loader, desc=f"Epoch {epoch +
31                                     1}/{config['num_epochs']}")
32
33         for inputs, labels in progress_bar:
34             inputs = inputs.to(device)
35             labels = labels.to(device)
36
37             optimizer.zero_grad()
38             outputs = model(inputs)
39             loss = criterion(outputs, labels)
40             loss.backward()
41             optimizer.step()
42
43             running_loss += loss.item() * inputs.size(0)
44             progress_bar.set_postfix(loss=loss.item())
45
46     epoch_loss = running_loss / len(train_loader.dataset)
47     train_losses.append(epoch_loss)

```

```

47     val_acc = evaluate(model, val_loader, device)
48     val_accuracies.append(val_acc * 100)
49
50     print(f"\nEpoch {epoch + 1}/{config['num_epochs']}")
51     print(f"Train Loss: {epoch_loss:.4f} | Val Acc: {val_acc:.2%}")
52
53     if val_acc > best_acc:
54         best_acc = val_acc
55         torch.save(model.state_dict(), "best_model.pth")
56
57     plot_training_curves(train_losses, val_accuracies)
58     plot_confusion_matrix(model, val_loader, class_names, device)
59     print(f"\nTraining complete. Best Val Acc: {best_acc:.2%}")

```

预测函数:

```

1 def predict(image_path, model, class_names):
2     model.load_state_dict(torch.load("best_model.pth"))
3     model.eval()
4     transform = transforms.Compose([
5         transforms.Resize(256),
6         transforms.CenterCrop(224),
7         transforms.ToTensor(),
8         transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224,
9             0.225])
10    ])
11    try:
12        image = Image.open(image_path).convert('RGB')
13    except FileNotFoundError:
14        print(f"Image file {image_path} not found.")
15        return None
16    image = transform(image).unsqueeze(0)
17    device = torch.device("cuda:0" if torch.cuda.is_available() else
18        "cpu")
19    image = image.to(device)
20    with torch.no_grad():
21        output = model(image)
22        _, pred = torch.max(output, 1)
23    return class_names[pred.item()]

```

批量重命名函数（用于验证分类结果）

```
1 def batch_rename_images(test_folder, model, class_names):
2     for filename in os.listdir(test_folder):
3         if filename.endswith('.jpg'):
4             image_path = os.path.join(test_folder, filename)
5             predicted_class = predict(image_path, model, class_names)
6             if predicted_class:
7                 number_part = ''.join(filter(str.isdigit, filename))
8                 new_filename = f"{predicted_class}-{number_part}.jpg"
9                 new_path = os.path.join(test_folder, new_filename)
10                os.rename(image_path, new_path)
11                print(f"Renamed {filename} to {new_filename}")
```

主函数:

```
1 if __name__ == "__main__":
2     print(f"CUDA available: {torch.cuda.is_available()}")
3     device = torch.device("cuda:0" if torch.cuda.is_available() else
4                           "cpu")
5
6     train_loader, val_loader, class_names = prepare_dataloaders()
7     print(f"Class names: {class_names}")
8
9     model = create_model(use_pretrained=True).to(device)
10    criterion = nn.CrossEntropyLoss()
11    optimizer = optim.Adam(model.parameters(), lr=config["lr"],
12                           weight_decay=0.0001)
13
14    train_model(model, criterion, optimizer, train_loader, val_loader)
15
16    test_folder = "./test"
17    visualize_predictions(test_folder, model, class_names, device)
18    batch_rename_images(test_folder, model, class_names)
```